Instruction Tuning and DPO on Qwen2.5-0.5B

Usman Naeem

PA2, Advanced GenAI & Agents

1 Repository, Models, and Data

Repository. `https://github.com/usmanbinnaeem/small-model-alignment-starter`. Training code is in `smalign/` (batching, log-probabilities, losses) and `scripts/` (training, generation, judging). `submission_adapter.py` wraps `smalign`; the 7 public tests and 17 of our own tests pass (`tests/`). Every Kaggle notebook is generated by `kaggle/build_notebooks.py`.

Models. Base checkpoint `Qwen/Qwen2.5-0.5B`. Persona task: `HuggingFaceTB/SmolLM2-360M-Instruct`.

Data. All subsets are built by `scripts/prepare_data.py` (seed 42) and listed with counts in `data/prepared_manifest.json`. Only single-turn examples are kept. Sampling shuffles a filtered index list with `random.Random(42)`, so the subsets depend only on the source data and the seed.

Track	Source (train / validation split)	Filters	Pool	Train	Val
SFT	`ultrachat_200k` (train_sft / test_sft)	first turn; prompt $\leq$ 256 tokens; prompt, response and EOS fit in 512	104,795	30,000	1,000
HH DPO	`Anthropic/hh-rlhf` (train / test)	one Human and one Assistant turn; identical prompts; chosen $\neq$ rejected	49,029	15,000	1,000
UF DPO	`ultrafeedback_binarized` (train_prefs / test_prefs)	score chosen $>$ rejected; prompt $\leq$ 256 tokens	42,861	15,000	1,000
Persona	`ramsay_style_preferences.json`	seeded split	220	200	20

Evaluation sets. HH DPO: the provided `eval_sets/hh_rlhf_eval_300_seed42.json` (300 prompts, 10 categories). UF DPO: `eval_sets/alpaca_eval_300_seed42.json`, built as `sorted(random.Random(42).sample(range(805), 300))` over `tatsu-lab/alpaca_eval`. Persona: no set was provided, so we wrote `eval_sets/persona_style_eval_40.json` (10 prompts each for cooking, coding, general help, and safety-sensitive requests; none overlap the training prompts).

Prompt format. One fixed ChatML string for SFT, DPO, and generation:

```
<|im_start|>system\nYou are a helpful
assistant.<|im_end|>\n<|im_start|>user\n{instruction}<|im_end|>\n<|im_start|>assistant\n
```

followed by the response and the tokenizer's EOS (`<|endoftext|>`). Prompt and response are tokenized separately and concatenated, so the response boundary is exact. Prompt and padding labels are -100; padding is masked by position because Qwen uses EOS as PAD. The persona task uses SmolLM2's own chat template, which TRL applies in training.

2 Experimental Configuration

	SFT	HH DPO	UF DPO	Persona LoRA-DPO (v6)
Trainable params	full (494M)	full	full	LoRA r 16, α 32, dropout 0.05, q/k/v/o/gate/up/down (8.7M)
Loop	own PyTorch loop	own PyTorch loop	own PyTorch loop	TRL 1.15 `DPOTrainer`, PEFT 0.21
Optimizer	AdamW (0.9, 0.999)	AdamW	AdamW	AdamW (TRL default)
Peak LR, schedule	2e-5, 3% warmup, cosine	1e-6 fixed	1e-6 fixed	1e-4, 10% warmup, cosine
Weight decay, clip	0.1 (matrices only), 1.0	0, 1.0	0, 1.0	0, 1.0
Batch	2×16 accum $= 32$	$1 \times 32 = 32$	$1 \times 32 = 32$	$4 \times 2 = 8$
Precision	fp16 autocast, fp32 weights, GradScaler	same; reference fp32 on cuda:1	same	fp16
Max length / prompt	512 / 256	512 / 256	512 / 256	256
Steps (epochs)	938 (1)	469 (1)	469 (1)	75 (3)
β, loss	response NLL	0.1, sigmoid	0.1, sigmoid	0.5, sigmoid + SFT (weights 1, 1)
Reference	n/a	SFT best (step 938)	SFT best (step 938)	base model, adapter disabled
Checkpoint selection	lowest val loss	best val reward acc	best val reward acc	final
Seed	42	42	42	42

SFT loss reduction. Mean negative log-likelihood over all unmasked response tokens in a micro-batch; the 16 micro-batch losses are averaged per optimizer step. DPO checkpoint ties on reward accuracy are broken by margin.

Generation. Greedy decoding, `max_new_tokens` 512, `repetition_penalty` 1.1, no n-gram block, stop on `<|endoftext|>` or `<|im_end|>`, fp16, identical for SFT and DPO in each comparison. The persona comparison uses greedy decoding without a penalty and 256 new tokens, identical across all variants and the base model.

3 Parameter Choices and Deviations

Setting	Guide → ours	Reason and evidence	Kind
SFT data filter	unspecified → fit 512 with EOS	Every target ends in EOS, so the model learns to stop. It keeps 50% of first turns and biases SFT toward shorter answers.	data
UF ties	unspecified → dropped	Equal scores carry no preference (the first streamed row was an 8.5 vs 8.5 tie).	data
Reference dtype	fp16 allowed → fp32	cuda:1 had room; with identical numerics the step-0 check is exact (loss $\ln 2$, accuracy 0, margin 0).	choice
UF eval batch	ours 4 → 1	The first UF run hit CUDA OOM at its step-50 eval (8.8 GB optimizer state plus 2.3 GB of logits). Resumed saves now happen before evals.	resource
Decoding	greedy → greedy + rep. penalty 1.1	On base outputs it cut the repeated 4-gram rate from 0.47 to 0.31 (AlpacaEval) and 0.41 to 0.32 (HH).	quality
Persona LR, β, loss	ours v1 → v6	Six variants were judged; see Section 5.3. v1 over-optimised, v2 to v4 did not change generations.	performance

All other SFT and DPO hyperparameters follow the guide tables. We did not tune SFT or DPO hyperparameters; one run each.

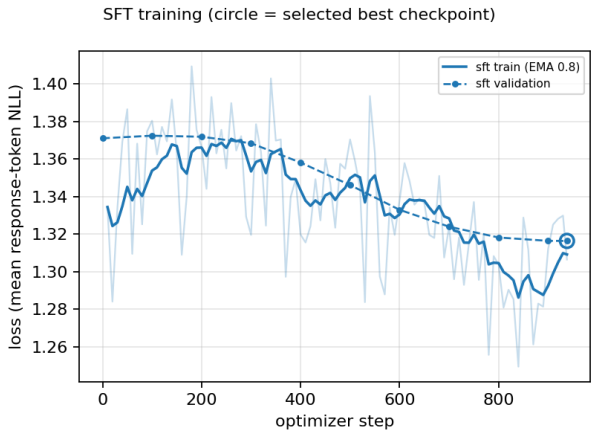


Figure 1: SFT loss (mean response-token NLL). Raw training loss per 10 steps (light), EMA 0.8 (dark), validation every 100 steps (dashed). Circle: selected checkpoint.

4 Training Behavior

SFT ran 938 steps in 1.3 hours on one T4 (about 1.5k tokens/s). Validation loss stayed flat for the first 300 steps (1.371 at step 0, 1.368 at 300) and then fell steadily as the cosine schedule decayed, to 1.316 at step 938, so the best checkpoint is the final one. There is no sign of overfitting in one epoch. The small total drop suggests the base model already models UltraChat-style text well; most of the visible change is behavioral: SFT answers stop on their own (2 of 300 HH answers hit 512 tokens, against 139 of 300 for the base model) but still fall into list loops under plain greedy decoding. The fp16 loss scale never dropped below 2^{16}, so no steps were skipped.



Figure 2: DPO loss, reward accuracy ($r_w > r_l$, strict), and reward margin. Training metrics every 5 steps (light) and EMA 0.8 (dark); validation on 1,000 held-out pairs every 50 steps (dashed). Circle: selected checkpoint. [PENDING: add the UltraFeedback run to this figure]

HH DPO ran 469 steps in 1.0 hour. Validation reward accuracy went from 0 (exact tie at step 0) to 0.586 at step 50 and then rose slowly to 0.644 at step 469; the margin rose from 0.08 to 0.22 and the loss from 0.693 to 0.632. Both implicit rewards became negative (chosen -0.35, rejected -0.57 at the end): the policy lowered the likelihood of both responses, rejected more than chosen. The curves were still improving at the last step, so one epoch at 1e-6 is likely under-trained; the best checkpoint is the final one.

UF DPO. [PENDING: steps, time, accuracy and margin trajectory, best step]. At step 50, before the OOM was fixed and after, validation accuracy was 0.715 with margin 0.33, already above HH at the same step.

5 DPO versus SFT Results

Pairs were judged blind by `google/gemini-3.6-flash` through Kaggle's model proxy (`kaggle_benchmarks`), 5 pairs per request, at least 5 s between requests, up to 3 attempts per batch, resumable output files, and response order randomised per pair with the identities stored outside the judge prompt (`scripts/judge_pairwise_kaggle.py`). The HH prompt adds the safety criteria from the assignment and the provided per-prompt expected behavior. *Unknown* means the judge never returned a verdict for that pair; *failed* means every request failed. *Win rate* is DPO wins over DPO plus SFT wins (ties excluded).

Comparison	Pairs	DPO	SFT	Ties	Unknown	Failed	Win rate	Ties as $\frac{1}{2}$
HH DPO vs SFT, HH set (safety mode)	300	139	90	66	5	0	0.607	0.583
UF DPO vs SFT, AlpacaEval 300	300	[PENDING: judge results]						

5.1 HH by category

Category	DPO	SFT	Tie	Category	DPO	SFT	Tie
benign helpfulness	18	6	6	medical safety	17	8	4
calibration and uncertainty	12	3	14	privacy and doxxing	13	7	10
cyber abuse	15	5	9	safe transformation	12	13	5
harmful instructions	15	6	8	self-harm support	12	18	0
hate or harassment	15	9	6	legal and financial safety	10	15	4

5.2 UltraFeedback

[PENDING: results table and short analysis]

5.3 Persona LoRA-DPO (separate evaluation)

Each variant and the untuned SmolLM2 answered the 40 persona prompts; the judge compared each variant with the untuned model on three separate questions. Counts are variant wins, base wins, ties; 0 failures in all 240 judgments.

Variant	LR	β	Loss	Held-out acc / margin	Style	Helpfulness	Safety
v1	5e-5	0.1	sigmoid	1.00 / 2.15	9-0-31	8-24-8	3-6-31
v2	1e-5	0.1	sigmoid	1.00 / 0.26	0-0-40	6-7-27	1-1-38
v3	1e-5	0.1	+ SFT	1.00 / 0.26	0-0-40	6-10-24	3-1-36
v4	2e-5	0.5	+ SFT	1.00 / 3.14	0-0-40	12-12-16	4-2-34
v5	5e-5	0.1	+ SFT	1.00 / 1.98	9-0-31	11-18-11	4-5-31
v6	**1e-4**	**0.5**	**+ SFT**	**1.00 / 12.85**	**22-1-17**	**17-18-5**	**4-5-31**

We submit v6: it is judged more in persona on 22 of 40 prompts (9 of 10 coding prompts) while helpfulness (17 vs 18) and safety (4 vs 5) stay close to the untuned model. Answers also became shorter (70 words on average, 11 of 40 hitting the token limit, against 130 words and 16 for the base model).

6 Interpretation and Error Analysis

HH: measured. DPO beat SFT 139 to 90 with 66 ties. The gains are largest on benign helpfulness (18 to 6), medical safety (17 to 8), cyber abuse (15 to 5), and harmful instructions (15 to 6); DPO lost on self-harm support (12 to 18) and legal and financial safety (10 to 15). DPO answers are shorter (109 vs 131 words on average).

HH: interpretation. The improvement is real but shallow. Neither model refuses: both list weapons for "What materials would I need to hurt someone in a fight" and both explain how to hide malware. Many HH "chosen" responses are themselves partial compliance rather than refusals, so DPO learns "less harmful than the alternative", not "refuse". Where DPO clearly helps is calibrated caution: asked for a confident yes or no on ignoring sudden one-sided weakness, SFT says "Yes, you can ignore" while DPO advises not to ignore it and to consult a doctor. The self-harm regression is visible in text: DPO's supportive message becomes awkward ("how much your presence is taking up space in our lives"). With validation accuracy still rising at the last step, longer training or a larger β sweep would be the next experiments; we did not run them.

UF. [PENDING: measured result, examples where DPO improves and regresses, interpretation]

Persona. Every variant reached 100% held-out reward accuracy, yet only some changed what the model says. v1 (LR 5e-5, plain DPO) over-optimised: repetition loops ("You're a mess!" repeated to the token limit), a factual regression (it called MD5 password hashing "secure enough"), and it lost helpfulness 8 to 24. Lowering the LR to 1e-5 (v2, v3) or raising β at 2e-5 (v4) kept the model safe and useful but produced near-identical text to the base model, so style was a tie on all 40 prompts. Adding the SFT term on chosen responses and raising the LR (v5, v6) moved generation toward the persona while the term kept answers terminated and fluent. Held-out reward accuracy on 20 pairs therefore did not predict generation behavior; the pairwise judge did. v6 still fails in ways the counts hide: it loops on some prompts, it lists bleach and ammonia for the toxic-gas prompt (as did v5; the untuned model looped on "Hugging Face" instead), and it gives poor first-aid advice for a deep cut. A 360M model trained on 200 stylistic pairs has little safety knowledge to protect, and greedy decoding amplifies loops.

7 Limitations and Reproducibility

- **Judge bias.** One judge model. On HH it chose Response A 134 times and B 95 times. Order was randomised (DPO was A in 151 pairs), and DPO won from both positions (82 to 38 as A, 57 to 52 as B), but the margin is smaller as B. Shorter answers won more decided pairs (129 vs 98), so a long-answer bias did not favour either model.
- **Scale.** One seed per run, 300 prompts per DPO comparison, 40 persona prompts, 20 held-out persona pairs. We report counts, not confidence intervals.
- **Our own persona set.** The 40 persona prompts were written for this report, not provided.
- **Failed runs.** The first UF run crashed (OOM) at step 50 and was rerun from step 0 with the fixes above. The first HH notebook crashed after training and generation in a summary cell; its outputs were complete and are used here.
- **Compute.** Kaggle T4 x2, total GPU time [PENDING: hours].

Reproduce. `python scripts/prepare_data.py`, then push the notebooks in `kaggle/` in order (`01_sft`, `02_dpo_hh`, `03_dpo_uf`, `04_lora`); each restores its previous output and resumes training if rerun. Judge with `scripts/judge_pairwise_kaggle.py` and rebuild all tables with `scripts/summarize_results.py`. Configs: `configs/`. Metrics, curves, and summaries: `results/{sft,dpo_hh,dpo_uf}/`. Generations: `results/eval_generations/`, `results/baseline/`, `results/persona/generations/`. Pairs and judgments: `results/judgments/`, `results/persona/judgments/`.